



Hands-On Activity: Use subqueries to refine data



Activity Overview

As you've learned, a subquery is a query that is nested inside of another query. The subquery filters or sorts data to prepare it to be used by the outer query to produce its final result. This allows data professionals to create more nuanced queries that provide specific insights from the data!

For example, perhaps a data analyst working in human resources has been asked to determine the average salary of employees working within a specific department. The analyst can use a subquery to first find the total salary and number of employees within the department. Then, the outer query will use these numbers to calculate the average salary within a department. It's a step-by-step process, each step relying on the one before it.

In this activity, you will practice using **SELECT** statements with **FROM**, **WHERE**, and **GROUP BY** clauses to build your subqueries. By the time you complete this activity, you will be

able to create a subquery using **SELECT** statements with **FROM**, **WHERE**, and **GROUP BY** clauses and analyze the query's results.

Scenario

Review the following scenario. Then complete the step-by-step instructions.

You work for an organization that is responsible for the safety, efficiency, and maintenance of transportation systems in your city. You have been asked to gather information around the use of Citi Bikes in New York City. This information will be used to convince the mayor and other city officials to invest in a bike sharing and rental system to help push the city toward its environmental and sustainability goals.

To complete this task, you will create three different subqueries, which will allow you to gather information about the average trip duration by station, compare trip duration by station, and determine the five stations with the longest mean trip durations.

Step-By-Step Instructions

Follow the instructions to complete each step of the activity. Then answer the questions at the end of the activity before going to the next course item.

Step 1: Access the dataset

For this activity, you will need to log in to your BigQuery account to use the public **new_york_citibike** dataset. Then, use the table called **citibike_trips**.

Note: Many of the public databases on BigQuery are living records and, as such, are periodically updated with new data. Throughout this course, if your results differ from

those you encounter in videos or screenshots, there's a good chance it is due to a data refresh.

Step 2: Create a subquery to find the average trip duration by station

First, find the average trip duration for a particular station. This subquery will be applied to the `SELECT` statement, but first you need to build the outer query:

1. Select **COMPOSE A NEW QUERY** from the SQL workspace in BigQuery. Or, if you have the `new_york_citibike.citibike_trips` table open, select **Query** from the table's menu to compose a new query.
2. In line 1, enter `SELECT`, then press **Enter/Return**.
3. In line 2, press **Tab**, then enter `subquery.start_station_id` followed by a **comma**. Then press **Enter/Return**.
4. In line 3, enter `subquery.avg_duration`, then press **Enter/Return**.

Lines 1–3 in the query should read as follows:

```
SELECT
  subquery.start_station_id,
  subquery.avg_duration
```

This `SELECT` statement is used to create the outer query. The fields identified in lines 2 and 3 allow the `SELECT` statement to function similarly to a table with an alias, e.g.

`SELECT alias.column_name1, alias.column_name2`. This relies on the subquery below to populate the Query results table.

5. In line 4, press **Backspace** to remove the indentation to align the text with the **SELECT** statement. Enter **FROM**, then press **Enter/Return**.
6. In line 5, press **Tab**, then enter an open parenthesis **[(]**. Then press **Enter/Return**.
7. In line 6, press **Tab**, then enter **SELECT**. Then press **Enter/Return**.
8. In line 7, press **Tab**, then enter **start_station_id** followed by a **comma**. Then press **Enter/Return**.
9. In line 8, enter **AVG(tripduration) as avg_duration**. Then press **Enter/Return**.

Lines 4–8 in the query should read as follows:

```
FROM
  (
    SELECT
      start_station_id,
      AVG(tripduration) AS avg_duration
```

The subquery is created by using another **SELECT** statement, located within a **FROM** clause. It is used to determine the average trip duration by station. The station is indicated by the **start_station_id**. The average is calculated using the **AVG** function and is given the alias **avg_duration** using **AS**. This will appear as a column title in the results table after you run the query.

10. In line 9, press **Backspace** to remove the indentation to align the text with the **SELECT** statement.
11. Still in line 9, enter **FROM**
bigquery-public-data.new_york_citibike.citibike_trips, then press **Enter/Return**.
12. In line 10, enter **GROUP BY start_station_id) as subquery;** Notice that this line closes the parentheses that opened in line 5.
13. In line 11, add **ORDER BY avg_duration DESC;**

Lines 9, 10, and 11 in the query should read as follows:

```
FROM bigquery-public-data.new_york_citibike.citibike_trips
GROUP BY start_station_id) as subquery
ORDER BY avg_duration DESC;
```

FROM will tell the subquery where to pull the data from. In this case, it's pulling from the **citibike_trips** table. **GROUP BY** will group the rows by columns. Notice how it has the alias, **subquery**. This is what links to the two fields designated to be the titles of the results columns, as indicated in the outer query.

The complete query should read as follows:

```
SELECT
    subquery.start_station_id,
    subquery.avg_duration
FROM
    (
        SELECT
            start_station_id,
            AVG(tripduration) as avg_duration
        FROM bigquery-public-data.new_york_citibike.citibike_trips
        GROUP BY start_station_id) as subquery
ORDER BY avg_duration DESC;
```

13. Select **RUN**. The results will appear in the **Query results** window.

Note: Your results should contain two columns: **start_station_id** and **avg_duration**, however, the specific rows may differ from what is depicted here due to periodic refreshes of the source tables on BigQuery.

Query results			SAVE RESULTS	EXPLORE DATA	
JOB INFORMATION			RESULTS	CHART	PREVIEW
Row	start_station_id	avg_duration			
1	3633	71800.0			
2	3040	38351.69230769...			
3	3590	23327.95028409...			
4	3017	22982.66666666...			
5	3649	15286.27756286...			
6	3042	14540.52892629...			
7	3044	13802.34808259...			
8	3596	11686.22916666...			
9	3036	11445.09090909...			
10	3041	11295.04886258...			
11	3470	7464.55555555...			
12	3631	7047.439539347...			
13	3650	6854.2213740458			

Results per page: 50 1 – 50 of 882

Take some time to examine the query results. In this case there are 882 lines of data that you can use to analyze and compare the average trip durations between each station. You will continue to work with the average trip duration per station in the next query.

Step 3: Create a subquery to compare trip duration by station

Create a new query to compare the average trip duration per station to the overall average trip duration from all stations. This will provide insights about how long people typically use the bikes that they get from a particular station in comparison to the overall average. This subquery will be applied to the FROM clause in a SELECT statement. To begin:

1. Select the **blue plus sign** tab to compose a new query.
2. In line 1, enter `SELECT`, then press **Enter/Return**.
3. In line 2, press **Tab**, then enter `starttime` followed by a **comma**. Then press **Enter/Return**.

4. In line 3, enter `start_station_id` followed by a **comma**, then press **Enter/Return**.
5. In line 4, enter `tripduration` followed by a **comma**, then press **Enter/Return**.

Lines 1–4 in the query should read as follows:

```
SELECT
    starttime,
    start_station_id,
    tripduration,
```

To begin the query, note there are three fields from which to gather data in order to request the average trip durations. Here, use `starttime` (a somewhat unique identifier for the results) to mark when trips from a particular station started; the `start_station_id`, which identifies the station ID for each trip; and the `tripduration`, which measures the length of each trip in seconds. These will be the first three columns in your results table.

You'll generate two additional columns in the results table:

`avg_duration_for_station` and `difference_from_average`. Begin with the subquery for `avg_duration_for_station`:

6. In line 5, enter an open parentheses `(`

7. In line 6, enter `SELECT ROUND (AVG (tripduration) , 2)`

8. In line 7, enter `FROM`

`bigquery-public-data.new_york_citibike.citibike_trips.`

8. In line 8, enter `WHERE start_station_id = outer_trips.start_station_id.`

9. In line 9, close the parentheses `)` that were opened in line 5, making sure that both align in indentation. Then, on the same line, name the column by typing `AS avg_duration_for_station,`

Lines 5–9 in the query should read as follows:

```
(
    SELECT ROUND(AVG(tripduration),2)
    FROM bigquery-public-data.new_york_citibike.citibike_trips
    WHERE start_station_id = outer_trips.start_station_id
) AS avg_duration_for_station,
```

A **SELECT** statement is used to begin a subquery that will return the average trip duration for each station. Notice that the **ROUND** function is used to round the numerical value of each trip duration to the nearest hundredth of a second. (Remember: trip duration is measured in seconds.) **FROM** indicates that you want the query to pull from the **citibike_trips** table within the **new_york_citibike** dataset inside the **bigquery-public-data** database. **WHERE** tells the query to link the **start_station_id** with the output of the query: a new column labeled **avg_duration_for_station**.

Now, begin the subquery to create the second new column in the results table: **difference_from_avg**.

10. In line 10, enter **ROUND(tripduration -** (. Make sure it begins on the same indentation level as the previous columns that you selected.

11. In lines 11, 12, and 13 enter:

```
SELECT AVG(tripduration)
FROM bigquery-public-data.new_york_citibike.citibike_trips
WHERE start_station_id = outer_trips.start_station_id), 2) AS
difference_from_avg
```

This closes the subquery that creates the **difference_from_avg** column. Now, what remains is to close out the outer query.

12. In lines 14, 15, and 16 outdent all the way back to the left and enter:

```
FROM bigquery-public-data.new_york_citibike.citibike_trips AS
outer_trips
```



```
ORDER BY difference_from_avg DESC
```

```
LIMIT 25;
```

Lines 10–16 in the query should read as follows:

```
ROUND(tripduration - (  
    SELECT AVG(tripduration)  
    FROM bigquery-public-data.new_york_citibike.citibike_trips  
    WHERE start_station_id = outer_trips.start_station_id), 2) AS  
difference_from_avg  
FROM bigquery-public-data.new_york_citibike.citibike_trips AS outer_trips  
ORDER BY difference_from_avg DESC  
LIMIT 25;
```

A second **SELECT** statement is used to return the difference between the specific station's average trip duration and the overall average trip duration. Notice that the **ROUND** function is used again to round to the nearest hundredth of a second. The average trip duration is gathered from the **citibike_trips** table using a second **FROM** clause. **WHERE** tells the query to link the **start_station_id** with the output of the query: a new column labeled **difference_from_avg**.

This will contain the difference between the overall average trip duration and the average trip duration per station. The third **FROM** clause pulls the data from the **citibike_trips** table and labels it as **outer_trips**. **LIMIT** is used to limit the output in the results table to 25 results. Depending on the amount of information you need from the dataset, you can change this number as needed to increase or decrease the results, or you can remove it completely to see all results.

Note: If you remove the limit, it may take longer for the query to run. You may also have to perform another query to find specific data.

The complete query should read as follows:

```
SELECT  
    starttime,  
    start_station_id,
```

```

tripduration,
(
    SELECT ROUND(AVG(tripduration),2)
    FROM bigquery-public-data.new_york_citibike.citibike_trips
    WHERE start_station_id = outer_trips.start_station_id
) AS avg_duration_for_station,
ROUND(tripduration - (
    SELECT AVG(tripduration)
    FROM bigquery-public-data.new_york_citibike.citibike_trips
    WHERE start_station_id = outer_trips.start_station_id), 2) AS
difference_from_avg
FROM bigquery-public-data.new_york_citibike.citibike_trips AS outer_trips
ORDER BY difference_from_avg DESC
LIMIT 25;

```

13. Select **RUN**. The results will appear in the **Query results** window.

Note: Your results should contain five columns: `starttime`, `start_station_id`, `tripduration`, `avg_duration_for_station`, and `difference_from_avg`, however, the specific rows may differ from what is depicted here due to periodic refreshes of the source tables on BigQuery.

Query results

SAVE RESULTS

EXPLORE DATA

JOB INFORMATION		RESULTS	CHART	PREVIEW	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	starttime	start_station_id	tripduration	avg_duration_for_sta	difference_from_avg		
1	2018-01-22T18:20:27.512000	3082	19510049	2061.72	19507987.28		
2	2018-02-21T14:15:10.932000	3349	15962256	2908.95	15959347.05		
3	2018-03-15T18:21:38.801000	3041	15020934	11295.05	15009638.95		
4	2018-02-21T15:30:02.388000	3042	13931824	14540.53	13917283.47		
5	2018-02-12T15:38:54.233000	3042	13586276	14540.53	13571735.47		
6	2018-03-11T03:52:30.296000	3383	12479323	1780.86	12477542.14		
7	2018-02-08T21:46:47.029000	3064	11749576	2566.28	11747009.72		
8	2018-01-28T03:51:28.614000	3537	11699746	3812.1	11695933.9		
9	2018-01-27T15:37:18.019000	501	11138807	1192.56	11137614.44		
10	2018-05-05T13:46:13.022000	343	10283682	1623.92	10282058.08		
11	2018-04-25T22:51:23.862000	514	10172645	1392.53	10171252.47		
12	2017-04-03T07:50:27	243	9735948	1126.81	9734821.19		
13	2017-10-04T18:23:14	83	9582723	1720.48	9581002.52		
14	2018-03-31T04:46:48.678000	362	9452933	948.42	9451984.58		
15	2017-10-07T16:23:15	3042	9340261	14540.53	9325720.47		
16	2017-04-04T18:48:05	545	9272394	860.0	9271534.0		
17	2017-04-02T16:29:23	387	9254340	1451.32	9252888.68		

Load more

Results per page: 501 - 25 of 25

14. Examine the results table. In the last column, **difference_from_avg**, there are some very large differences from the average duration, indicating that these stations have some significant outliers. It would probably be worthwhile to examine that further.

15. To continue with this activity, you can **save** this query so you can come back to it later, then **create a new query**. Alternatively, you can overwrite the existing query.

Step 4: Create a subquery to determine the five stations with the longest mean trip duration

Now, you'll compose a new query to filter the data to include only the trips from the five stations with the longest mean trip duration.

Write the following code and consider what it's doing:

```
SELECT
    tripduration,
    start_station_id
FROM bigquery-public-data.new_york_citibike.citibike_trips
WHERE start_station_id IN
    (
        SELECT
            start_station_id
        FROM
            (
                SELECT
                    start_station_id,
                    AVG(tripduration) AS avg_duration
                FROM bigquery-public-data.new_york_citibike.citibike_trips
                GROUP BY start_station_id
            ) AS top_five
        ORDER BY avg_duration DESC
        LIMIT 5
    );
```

Lines 1–4: This query begins by identifying the columns that will appear in the query results table: `tripduration` and `start_station_id`.

However, you don't want all of the records to be selected. You only want those records where the `start_station_id` matches one of the top five stations with the greatest average `tripduration`. To find this, you need a subquery.

Line 5: This is the beginning of the subquery. It uses an `IN` operator to filter records based on whether the `start_station_id` is in a list of `start_station_ids` produced by the subquery.

Lines 6–19: This is an inner subquery. It creates a derived table called `top_five` by performing the following steps:

1. It selects `start_station_id` from the main table.
2. For each `start_station_id`, it calculates the average `tripduration` and labels this average as `avg_duration`.
3. It groups the results by `start_station_id`, meaning the average trip duration is calculated separately for each unique `start_station_id`.
4. From this grouped and aggregated data, it orders the results by `avg_duration` in descending order.
5. It limits the results to the top five. In other words, it only keeps the records for the five `start_station_ids` with the highest average trip durations.

The result of the entire query is a list of records from the main table—specifically the `tripduration` and `start_station_id` for each record, but only those for records where the `start_station_id` is among the five stations with the greatest average trip durations. If you examine the query results, you will discover that only five of the `start_station_id` values are listed in column two. The output of the query should appear similar to this:

Note: Your results should contain two columns: `tripduration` and `start_station_id`, however, the specific rows may differ from what is depicted here due to periodic refreshes of the source tables on BigQuery.

Query results [SAVE RESULTS](#) [EXPLORE DATA](#)

JOB INFORMATION	RESULTS	CHART	PREVIEW	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	tripduration	start_station_id				
1	2548	3649				
2	4457	3649				
3	1961	3649				
4	1591	3649				
5	1084	3649				
6	1498	3649				
7	207	3649				
8	662	3649				
9	674	3649				
10	913	3649				
11	178	3649				

Results per page: 50 1 – 50 of 1768

Column one is `tripduration`; column two is `start_station_id`.

Congratulations! You successfully built and ran three different sets of queries with subqueries.

1. Reflection

1 / 1 point

The last query you ran returned every record from only the top five stations with the highest average trip durations. How would you modify the previous query to find all the records from the five stations with the *lowest* average trip durations?

- ☒ Change `DESC` to `ASC` in line 17
- ☐ Change `5` to `-5` in line 18
- ☐ Change `top_five` to `bottom_five` in line 16
- ☐ Change `AVG(tripduration) AS avg_duration` to `MIN(tripduration) AS min_duration` in line 13

✔ Correct

2. In the text box below, write 2-3 sentences (40-60 words) in response to each of the following questions:

1 / 1 point

- How do subqueries help you work with large datasets?
- Which other ways could you use subqueries to analyze data?

In the large dataset I can use a nested query to fulfill my analysis in the formation of a single query and get an effective, informative result. We can use the subqueries at the beginning or after the select statement or using the open and close parentheses after using the select statement. In this type we can use subqueries to analyze the data.

Feedback

In this activity, you created queries and subqueries to compare and analyze specific sections of a large dataset. Effective responses to the reflection questions might include:

- Learning how to create queries and subqueries is a key component of an analyst's ability to research and deliver useful findings to leadership or team members.
- Subqueries help you identify and work with smaller sections of data from a large dataset.
- Subqueries can be used to complete multiple operations at one time rather than creating multiple queries. They can be used to compare, sort, order, group, and aggregate, among many others.

Subqueries can be challenging—they take practice and repetition to execute well. Now that you have a starting point, take some time to practice working with different datasets within the bigquery-public-data database and building out your own subqueries.